# Lambe JSON mode reference

Lambe 0.9.6 · Custom-mode schema 1 · 15 October 2026

For people who want to edit a mode in a text editor, share definitions, or understand the implementation. Created and developed by André I. Ben, Lambe Labs.

The examples and behaviour below follow this release's Dart source. A writing mode assigns meanings and attachment rules to glyphs; it does not translate between languages. Glyph tokens identify shapes, not universal sound values.

## Contents

1. Choose the right file
2. Start with a complete importable mode
3. Top-level fields
4. Associations
5. Marks and positions
6. Contextual rules
7. Invisible input and R classification
8. Matching and inherited English behaviour
9. Reference data and JSONC
10. Files, storage and built-in changes
11. Validation and troubleshooting
12. Glyph token catalogue
13. Source map and further reading

**Illustrated edition | 18 October 2026**

Visual A: Mode structure and engine selection / page 2

Visual B: Mark placement and reading order / page 6

Visual C: Matching and invisible input / page 10

Visual D: Expressions and import workflow / page 15

Visual E: Core base tokens at a glance / page 16

VISUAL A

# A mode is a set of instructions

The JSON stores input mappings and behaviour switches. The renderer turns the resulting base and mark tokens into visible glyphs.

## 1 From an association to a written group

```
{"input":"t", "token":"tinco",
 "marks":[{"token":"a", "direction":8, "dx":0}]}
```




This is a structural example, not a complete linguistic mode. A token names a shape; the mode decides what that shape represents.

## 2 The outer object selects the engine

| | |
|---|---|
| `referenceData is not null` | **Reference engine** — First choice |
| `Otherwise: english is true` | **English engine** — Built-in English logic |
| `Otherwise` | **Empty engine** — Custom mappings |

A copied reference mode needs its full referenceData snapshot. The outer vowelsForward and autoR switches control attachment and contextual R.

# 1 Choose the right file

| File or data | Shape | Used by |
|---|---|---|
| Exported `*.tengwar-mode.json` | One object with `schemaVersion`, `id`, `name`, `associations`, `rules` | **Import mode** and **Export mode** |
| `modes/custom_modes.json` | `{"version":1,"modes":[...]}` | The app's saved collection; not the single-mode importer |
| `assets/modes/arabic-romanized-lambe.json` | One native custom-mode object | Bundled Arabizi default |
| `assets/modes/*.jsonc` | Reference map, words, preprocess | Readable source references; compiled definitions drive the built-in reference modes |
| A custom mode's `referenceData` | A comment-free reference definition nested inside a native mode | Editable copies of Quenya, Sindarin, Beleriand and German |
| Editor document JSON | Canvas/source data including `customModes` | Internal document model; not a mode import |

Use **Export mode**, edit a backup, then **Import mode**. Import validates the file and gives it a new `custom_...` ID, preserving existing saved modes. A `.jsonc` file cannot be made importable merely by renaming it to `.json`.

Native mode files use strict UTF-8 JSON: double-quoted keys and strings, real booleans, no comments, no trailing commas. Top-level unknown properties are not preserved on export. Definitions contain data, not executable user code.

# 2 Start with a complete importable mode

Save this whole block as `practice.tengwar-mode.json`, then choose **Import mode**. This deliberately small teaching mode defines only the inputs shown.

```
{
  "schemaVersion": 1,
  "id": "custom_practice",
  "name": "Practice",
  "english": false,
  "autoFlip": true,
  "autoR": true,
  "vowelsForward": true,
  "explicitCarrier": true,
  "carrier": "telco",
  "associations": [
    {"input": "b", "token": "umbar"},
    {"input": "m", "token": "malta"},
    {"input": "r", "token": "oore"},
    {"input": "t", "token": "tinco"},
    {"input": "a", "token": "a", "isMark": true, "isVowel": true, "direction": 8}
  ],
  "rules": [
    {
      "input": "m",
      "scope": "end",
      "baseToken": "malta",
      "marks": [{"token": "dotBelow", "direction": 2, "dx": 0}]
    }
  ]
}
```

Expected readings of the generated structure:

| Input | Output structure |
|---|---|
| ab | umbar with a above; the vowel attaches forward |
| ba | umbar, then telco with a above |
| ra | roomen, then the vowel carrier; R sees a vowel input |
| rb | oore, then umbar |
| mb | Plain malta, then umbar |
| bm | umbar, then malta with a dot below |
| a^b | A carrier with a, followed by umbar |

The final-M dot is an invented practice convention, not a general Tengwar rule. Change vowelsForward to false to attach a to the preceding base: ba then becomes one group, while ab begins with a carrier.

For English, Quenya or Sindarin, duplicate the corresponding mode in the app and export that copy. A complete exported definition preserves details a minimal example omits.

# 3 Top-level fields

Defaults apply when a field is absent or null, except where required. Use the exact spelling and case of every key.

| Field | Type / default | Effect |
|---|---|---|
| schemaVersion | Number / 1 | Missing is accepted; an explicit value other than 1 is rejected |
| id | Required string | Must begin with custom_; importer assigns a fresh ID after validation |
| name | Required string | Must contain non-whitespace text |
| english | Boolean / false | Selects the English engine when referenceData is null |
| referenceData | Object / null | Complete copied reference definition; takes priority over english |
| hebrewDefaults | Boolean / false | In the Empty engine, normalises ’ and ʕ to apostrophe and uses Hebrew-style pending-mark spacing; does not install a Hebrew alphabet |
| autoFlip | Boolean / true | Allows automatic turned S/Z forms in the renderer |
| autoR | Boolean / true | Enables contextual R unless r_context is in removedRules |
| vowelsForward | Boolean / true | Ordinary mark associations attach to the following base; false uses the preceding base where possible |
| explicitCarrier | Boolean / true | Enables literal ^ as a carrier control; invisible controls are removed before this stage |
| carrier | Base token / "telco" | Base used when marks have no suitable host |
| associations | Object array / [] | Literal input-to-output mappings |
| rules | Object array / [] | Contextual outputs with precedence over associations |
| disabledRules | String array / [] | Temporarily disables named inherited English groups |
| removedRules | String array / [] | Records removed English groups; also recognises r_context |
| deletedInputs | String array / [] | Blocks deleted inherited inputs and makes matched text literal |

Engine choice is `referenceData != null` → reference engine; otherwise `english == true` → English engine; otherwise Empty engine. `english: true` retains English built-in logic even if `associations` is empty. Conversely, `english: false` does not inherit English shorthands or pronunciation exceptions.

In a reference copy, the outer `vowelsForward` and `autoR` values govern attachment and R. Editing only the nested `tehtarFollow` or `rrule` does not update those outer switches. Duplicate through the app to obtain their initial matching values.

# 4 Associations

Each association matches literal lower-case input with no whitespace. It can consume one character, a digraph, a symbol or a longer sequence. The source is lowercased before matching; there is no separate uppercase alphabet.

| Field | Type / default | Effect |
|---|---|---|
| `input` | Required string | Non-empty, lower-case, no whitespace; unique within `associations` |
| `token` | Required string | Registered base or mark token; empty string for no-output entries |
| `isMark` | Boolean / `false` | True emits a mark; false emits a base |
| `isVowel` | Boolean / inferred | Defaults true only when `isMark` is true and input is exactly `a`, `e`, `i`, `o` or `u` |
| `direction` | Number / 8 | Keypad anchor for a mark association; use an integer from 1 to 9 |
| `marks` | Mark array / `[]` | Extra marks on a base association; not allowed on mark-only associations |
| `inherited` | Boolean / `false` | True identifies an entry delegated to inherited English/reference logic |
| `enabled` | Boolean / `true` | A disabled selected association leaves its input literal |
| `noOutput` | Boolean / `false` | Consumes input without a glyph or a space |
| `classifiesPrevious` | Boolean / `false` | With `noOutput`, changes the preceding input's R classification |

`isMark` describes the output shape; `isVowel` classifies the input for R look-ahead. They are independent. A consonant-modifying tehta can have `isVowel: false`; a full-letter vowel can have `isMark: false, isVowel: true`. Set `isVowel` explicitly for `y`, accented vowels and multi-letter inputs.

To create a base with a mark, put this object in `associations`:

```
{
  "input": "nt",
  "token": "tinco",
  "isMark": false,
  "isVowel": false,
  "inherited": false,
  "marks": [{"token": "nasalBar", "direction": 8, "dx": 0}]
}
```

Set `inherited: false` when editing an inherited entry by hand. Changing only its token while leaving it inherited may leave the built-in interpretation in effect. The graphical editor sets this flag when an inherited association is edited or disabled.

To restore a deleted input, remove it from `deletedInputs` and add/enable its association. Avoid duplicate JSON object keys: a JSON decoder can discard an earlier value before Lambe validates the result.

VISUAL B

# Where a mark goes

Position, horizontal displacement and reading order are separate choices. The JSON fields below control different parts of the result.

## 1 direction selects a keypad anchor

| | | |
|---|---|---|
| 7 | 8 | 9 |
| 4 | 5 | 6 |
| 1 | 2 | 3 |

```
8 = above     2 = below

4 = left      6 = right

5 = centre / inside
```

Native marks default to direction 8, including a token named dotBelow. Set direction 2 to place that mark below.

```
{"token":"dotBelow", "direction":2, "dx":0}
```

## 2 vowelsForward changes attachment

### true: following base

input at



**Read a, then t**

### false: preceding base

input ta



**Read t, then a**

The line still runs left to right. direction does not change reading order. dx adds horizontal displacement; at layout step 34, dx: 6.8 equals one 0.2-unit move right.

# 5 Marks and positions

Every item in a `marks` array is a `ModeMark`:

```
{"token": "dotBelow", "direction": 2, "dx": 0}
```

`token` is required. `direction` defaults to 8; `dx` defaults to 0 and must be finite. Both are numeric; use integer directions. Positions follow the number pad:

| Direction | Anchor | Direction | Anchor | Direction | Anchor |
|---|---|---|---|---|---|
| 7 | Upper left | 8 | Above | 9 | Upper right |
| 4 | Left | 5 | Centre/inside | 6 | Right |
| 1 | Lower left | 2 | Below | 3 | Lower right |

`direction` selects an anchor; `dx` adds horizontal displacement in the layout coordinate system. It is not a count of key presses. At the default layout step of 34, `dx: 6.8` corresponds to one 0.2-unit movement to the right. A native mark object does not store `dy`, rotation or a multi-step `path`; those are properties of generated canvas objects. The canvas movement controls use 0.2 units per press.

Native JSON defaults to direction 8 even for a token named `dotBelow`. Set direction 2 yourself. The reference expression interpreter instead calls the glyph library's default placement for each mark. Naming a mark and positioning it are separate operations.

Several marks may share a base. Engines can also add spacing to pending marks; consequently, a stored `dx: 0` does not guarantee that repeated vowels overlap. Final-S curls receive the release's shared additional horizontal offset of 0.2 units during layout.

# 6 Contextual rules

A rule consumes its literal `input` once. Its output is not fed back through input matching. Rules use a single scope; they are not regular expressions.

| Field | Type / default | Effect |
|---|---|---|
| `input` | Required string | Non-empty lower-case literal text without whitespace |
| `scope` | String / `"anywhere"` | One of the scopes below |
| `context` | String / `""` | Comma-separated alternatives for `before` or `after` |
| `baseToken` | Base token / `"telco"` | Base emitted unless marks-only or previous-base output is selected |
| `marksOnly` | Boolean / `false` | Emit marks using the mode's attachment direction |
| `previous` | Boolean / `false` | Attach marks to the preceding generated base; no new base unless a carrier is needed |
| `marks` | Mark array / `[]` | Required non-empty when `marksOnly` or `previous` is true |
| `enabled` | Boolean / `true` | False excludes this rule from matching |

| Scope | Match condition |
|---|---|
| `anywhere` | At any position |
| `whole` | Input occupies the whole word, allowing terminal `. , ; : ! ?` |
| `start` | Input begins at index zero of the engine's current word |
| `end` | Input ends the word, allowing the same terminal punctuation |

| Scope | Match condition |
|---|---|
| `before` | The following text begins with one `context` alternative |
| `after` | The preceding text ends with one `context` alternative |

Context alternatives are trimmed but not automatically lowercased. Use lower-case strings, for example `"a,e,i,o,u"`. Context is inspected, not consumed. Scopes do not provide arbitrary combinations of conditions.

For a final-S mark on the preceding base, add this object to `rules`:

```
{
  "input": "s",
  "scope": "end",
  "marksOnly": true,
  "previous": true,
  "marks": [{"token": "finalS", "direction": 6, "dx": 0}]
}
```

With `previous: true`, previous-base attachment wins regardless of `marksOnly`. Otherwise, marks-only output queues for the next host in a forward mode or uses the previous host in a backward mode. A missing host causes a carrier to be supplied. For base-plus-marks output, keep both flags false.

Word boundaries differ between engines: the Empty engine starts with whitespace-separated chunks, while the reference engine splits standard punctuation separately. Do not assume parentheses, apostrophes or hyphens behave identically in every mode. Test the exact punctuation you intend to support.

# 7 Invisible input and R classification

No-output associations are processed before lexical transcription. They can consume an editing control without affecting shorthands, final forms or attachment as an ordinary visible character would.

Add this object to `associations` to make ^ classify the preceding input as a vowel:

```
{
  "input": "^",
  "token": "",
  "noOutput": true,
  "classifiesPrevious": true,
  "isVowel": true,
  "isMark": false,
  "marks": [],
  "inherited": false,
  "enabled": true
}
```

In the Practice mode, `rb` begins with `oore`, while `rb^` begins with `roomen`: the suffix classifies `b` as a vowel input for R. It adds no glyph and does not change `b` into a vowel sign. `r^b` classifies `r`, not `b`, so does not produce the same effect.

Set `isVowel: false` to classify the preceding input as a consonant. For a plain invisible separator/control, set `classifiesPrevious: false` and `isVowel: false`. An invisible entry with `classifiesPrevious: false, isVowel: true` supplies a vowel look-ahead override at its position instead.

The preceding unit is the input sequence recognised by the association scan, which can be a digraph, rather than necessarily one character. Whitespace and `. , ; : ! ? -` reset preceding-unit tracking. `rb ^` therefore does not classify `b`.

Validation requires a no-output entry to have `token: ""`, `isMark: false` and no marks. `classifiesPrevious` is only valid with `noOutput`. A no-output ^ is removed before the explicit-carrier stage, so the two uses of that symbol must be chosen deliberately.

In an Empty mode, the explicit-carrier check occurs before ordinary associations, so disable `explicitCarrier` if you want an ordinary visible association for ^. Custom rules run before that check. English/reference overrides have their own earlier handling of associations.

# 8 Matching and inherited English behaviour

At an ordinary input position:

1. Choose the longest matching enabled custom rule; equal lengths use the first rule in the array. There is no automatic preference for a more specific scope.

2. If no rule applies, select the longest association. A disabled longer association still blocks a shorter one and yields literal input. `deletedInputs` participates in the selection as disabled input.

3. In an English/reference copy, use inherited handling where a custom override has not handled the input. Inherited associations act as editor representations of those defaults.

4. An unmatched input in an Empty mode becomes literal text, apart from the engine's punctuation/whitespace handling.

A shorter custom rule can beat a longer association, because rules are a separate earlier stage. A longer inherited association can also mask an edited shorter English association, for example an inherited `sh` beside an edited `s`. Use an exact rule when you need an unambiguous override.

English includes word-level shorthands and spelling exceptions, so the list above describes local matching rather than every earlier lexical stage. The engine suppresses relevant shorthand handling when overrides/deletions would conflict. Test whole words and phrases, not only isolated letters.

The accepted English group IDs are:

| ID | Behaviour controlled |
|---|---|
| `shorthands` | Whole-word abbreviations such as the, of, of the, and, we |
| `silentE` | Silent-E dot |
| `doubleVowels` | Repeated-vowel handling |
| `diphthongs` | Vowel combinations |
| `consonantalY` | Consonantal Y |
| `labialisation` | QU/QW/KW |
| `nasals` | Nasal combinations |
| `th` / `ch` / `gh` | Contextual digraph readings |
| `ck` | CK combination |
| `doubles` | Doubled consonants |
| `x` | X combination |
| `finalS` | Final-S curl |
| `softC` | Soft C |
| `voicedS` | Voiced-S exceptions |

For example, `"disabledRules": ["shorthands", "finalS"]` disables those groups while retaining English elsewhere. `removedRules` also hides removed groups from the editor. Either list disables the

VISUAL C

# How input becomes output

Custom rules run before ordinary association matching. An invisible marker can change classification without adding a visible object.

## 1 At one input position

**Enabled custom rule**

Longest match; equal length uses array order

↓

**Association / deleted input**

Longest input; a disabled selection stays literal

↓

**Inherited handling or literal text**

English/reference defaults, or unmatched Empty input

This shows local matching. English word shorthands and reference preprocessing also have earlier stages, so test complete words and phrases.

## 2 An invisible suffix affects contextual R

`rb`

`oore + umbar`

b is a consonant input

`rb^`

`roomen + umbar`

^ classifies b as a vowel input

For the Practice mode: noOutput: true consumes ^; classifiesPrevious: true applies isVowel: true to b. The b still produces umbar. The marker itself adds no glyph or space.

---

Rule scope is a separate test: end matches a word ending; before / after inspect context without consuming it. It is not an arbitrary regular expression.

named English group. `r_context` is special: put it in `removedRules` or set `autoR: false`; putting it only in `disabledRules` does not turn R off.

These switches are implemented behaviours, not names of arbitrary custom rules. Unknown names do not create new features. English inherited processing remains ordered Dart logic, rather than a generic priority-sorted JSON rule list.

# 9 Reference data and JSONC

## A complete reference-backed custom mode

This is a small, importable demonstration of a native wrapper around reference data. It is not a complete linguistic mode.

```
{
  "schemaVersion": 1,
  "id": "custom_reference_practice",
  "name": "Reference practice",
  "english": false,
  "autoR": false,
  "vowelsForward": true,
  "explicitCarrier": false,
  "referenceData": {
    "name": "reference-practice",
    "normalizeVowels": false,
    "tehtarFollow": true,
    "rrule": false,
    "preprocess": {"æ": "ae"},
    "words": {"demo": "{ando}[triple-dot-above]{malta}"},
    "map": {
      "b": "{umbar}",
      "m": "{malta}",
      "t": "{tinco}",
      "ae": "[triple-dot-above]{yanta}",
      "^ng": "{nwalme}",
      "ng$": "{nwalme}",
      "ng": "[bar-above]{ungwe}",
      "â": "[triple-dot-above]{aara}"
    }
  },
  "associations": [],
  "rules": []
}
```

`at` gives a mark above tinco; `ta` ends with a short carrier. `ae` and `æ` give the same group. `demo` uses its direct output exception. A raw JSONC source contains the inner reference object, with optional comments, rather than this outer native wrapper.

## Reference fields Lambe reads

| Inner field | Behaviour in Lambe 0.9.6 |
|---|---|
| `map` | Required object of non-empty string keys to string output expressions |
| `words` | Optional whole-word replacement object |
| `preprocess` | Optional ordered regular-expression replacement object |
| `normalizeVowels` | Optional spelling normalisation when exactly true |
| `tehtarFollow` | Default attachment direction for a built-in reference mode; copied into outer `vowelsForward` when duplicated |

| Inner field | Behaviour in Lambe 0.9.6 |
|---|---|
| `rrule` | Built-in contextual-R flag; copied into outer `autoR` when duplicated |

Metadata such as `name`, `languageCode` and `info` can remain inside the snapshot, but is not used by this interpreter to assign glyphs. Other upstream options, including `suppressIntrawordDash`, are not implemented as independent switches here. The bundled preprocess table may still change hyphens.

## Map keys and output expressions

Map keys are literal strings with optional leading `^` and trailing `$` word anchors. They are not full regexes. Write them in lower case: input is lowercased, but map keys are not automatically lowercased. At each position, the longest consumed input wins; a tie prefers more anchors, then insertion order. Lambe does not enforce the upstream seven-character map-key guideline.

Use `{tinco}` for a base and `[acute]` for a mark. An expression can contain multiple bases and marks. Ordinary text outside these expressions is not emitted as literal text by `emit`; do not use plain replacement spelling in a `map` value.

A mark after a base in the same expression attaches to that base. A mark before an explicit base is queued for that base. A marks-only expression follows mode attachment, except that below-positioned marks and final-S attach backwards where possible. `{}` flushes pending marks to a carrier and introduces an attachment barrier. `{hook}` is a supported special spelling for a final-S mark; `{dash}` emits a literal hyphen.

Lambe's reference aliases are listed in the catalogue appendix. The interpreter accepts `alt-silme-nuquerna` as `silme-nuquerna`. Native associations must use the registered native spelling instead.

If a map has no match, `a/e/i/o/u/y` have fallback marks, and contextual R can supply `roomen/oore`. Apostrophe, curly apostrophe and middle dot form attachment barriers; a hyphen is emitted literally if preprocessing has not changed it. Other unmatched characters remain literal. This fallback does not make an incomplete reference map into a complete language mode.

## Preprocessing and exceptions

The engine lowercases source text, optionally normalises vowels, then applies `preprocess` entries in insertion order across the full input, before word segmentation. Values produced by an earlier replacement can be matched by a later one. In a preprocess regex, `^` and `$` refer to the whole input, unlike map-key word anchors.

Each preprocess key is a Dart `RegExp` pattern. Optional surrounding slashes are stripped only when both the first and last characters are `/`; flags such as `/pattern/i` are not supported. Values support `$1`, `$2`, and other numbered captures. Double backslashes inside JSON strings, for example `"/(\\w)\\.(\\w)/": "$1·$2"`. Use patterns that consume at least one character: zero-length matches are unsafe in this release's source-range tracker. A replacement must refer only to capture groups that exist.

With `normalizeVowels: true`, acute vowels `á é í ó ú ý` become circumflex forms; `ä ë ï ö ü` lose their diaeresis; `aa ee ii oo uu` become circumflex long vowels. Do not assume macrons, grave accents or decomposed Unicode sequences are normalised. Use explicit mappings/preprocessing for additional spellings.

`words` keys are lowercased and preprocessed before lookup. A value containing `{...}` or `[...]` is emitted as direct output; otherwise it becomes replacement input, is preprocessed, then transcribed. Whole-word replacement is a single stage, not recursive dictionary expansion. A non-inherited association or enabled custom rule found in the word suppresses its inherited `words` exception, letting custom output take effect.

The snapshot validator checks the shapes of `map`, `words` and `preprocess` and compiles preprocess regexes. It does not comprehensively check expression glyph names, attachment semantics, unknown options or capture-group references. A successful import is therefore only the first validation step.

### Editing a copied Quenya or Sindarin mode

1. Duplicate the built-in mode and export the copy.

2. For a simple change, edit/add a native association with `inherited: false`, or add an exact custom rule.

3. For a complex anchored mapping, preprocess stage or word exception, edit the nested `referenceData` object.

4. Preserve the outer switches and full snapshot. Import, select the imported mode, then test contrasting words.

Simple inherited associations exposed by the editor are not the whole reference definition. Removing `referenceData` discards its complex mappings and exceptions, even if the visible associations look plausible.

## 10 Files, storage and built-in changes

### The saved collection

`custom_modes.json` wraps complete definitions in a collection. The following is a complete, valid collection containing one empty mode; it is for understanding storage, not **Import mode**:

```
{
  "version": 1,
  "modes": [
    {
      "schemaVersion": 1,
      "id": "custom_empty_example",
      "name": "Empty example",
      "associations": [],
      "rules": []
    }
  ]
}
```

On writable Windows installations, storage is resolved relative to `Lambe.exe` in its `modes` directory, independent of the current working directory. If that directory is not writable, Lambe uses its application-support directory. Android uses application-support storage.

The app writes the collection through a temporary file and retains a previous valid `.bak` where available. A malformed primary collection can be recovered from that backup. Prefer export/import over editing the live store; if direct editing is necessary, close Lambe and keep both files first. Changes made while the app is open can be overwritten.

The store seeds missing copies of the five bundled mode files into the modes directory. It does not scan every JSON file there to load custom modes. In particular:

- The bundled Arabizi default is decoded from the packaged asset, then merged with saved modes by ID. Saved definitions win when IDs coincide.

- The four built-in reference modes read `lib/engine/reference_mode_data.dart`.

- Editing a seeded `quenya.jsonc`, `sindarin.jsonc` or Arabizi file beside the executable does not replace the active built-in definition. Import native custom definitions or update/rebuild the source as appropriate.

### Maintaining built-in source data

In the shared Flutter project, readable JSONC sources are in `assets/modes/`; English and Hebrew defaults are Dart implementations. To change a built-in reference mode, update its readable source and the matching comment-free object in `reference_mode_data.dart`, preserve both consistently, and rebuild the Windows and Android packages from the shared project. Copying an asset alone does not regenerate the embedded map. This release has no documented automatic reference-data regeneration command.

Changing `arabic-romanized-lambe.json` changes the packaged native default after rebuilding. An already-saved mode with that same ID can override it on an existing installation, so test a fresh profile as well as an upgraded one.

### Digits and Arabizi

The editor controller gives enabled digit-bearing native associations/rules a chance to use digits as letters, as Arabizi does. Otherwise, digits follow the **tengwar numerals** switch. In a digit-input mode, type a backslash before a number run, for example `\2026`, to force number treatment; the backslash is consumed. In a JSON string the same source text is written `"\\2026"`.

Numerals retain input digit order; the switch does not convert decimal values to base twelve. Digit handling and CR/LF separation occur in the controller before mode transcription. A direct engine-only test does not exercise those controller features. Digit keys present only inside `referenceData.map` are not detected as native digit-bearing associations by the controller.

Mode definitions do not include the current source sentence, canvas adjustments, a font file or font-studio work. Keep those separately as required.

## 11 Validation and troubleshooting

For a native file, validate by importing, selecting the new copy and inspecting both isolated groups and real words. Include word beginnings/endings, punctuation, consecutive vowels, an unattached vowel, a repeated consonant, mixed case, line breaks and unknown input. For reference copies, test every changed anchored mapping, preprocess expression and word exception.

| Symptom | What to check |
|---|---|
| Import rejects the file | Strict JSON syntax; one mode rather than a collection; schema version; `custom_` ID; non-blank name |
| Unknown glyph | Exact case-sensitive token in the catalogue; base versus mark category |
| Changed English entry has no effect | `inherited: false`; longer inherited association; earlier custom rule; relevant English group |
| Deleted input returns | Put inherited deletion in `deletedInputs`, not only remove the visible association |
| A disabled long input still blocks a short one | Disabled association selection deliberately preserves that matched input literally |
| A vowel uses the wrong host | Outer `vowelsForward`, explicit bases in an expression, `previous`, barriers and carriers |
| R changes unexpectedly | `autoR`, `removedRules`, `isVowel`, invisible suffixes and inherited spelling logic |
| Mark appears above instead of below | Native `direction` defaults to 8; specify 2 |
| A JSONC option has no effect | Check the interpreter's supported subset and the embedded data actually in use |

VISUAL D

# Reference expressions and files

Native mode JSON and nested reference expressions use different syntax. Keep the complete mode wrapper when importing an editable reference copy.

## 1 Two spellings for the same output pieces

NATIVE ASSOCIATION

```
{"input":"t", "token":"tinco",
 "marks":[{"token":"a", "direction":8, "dx":0}]}
```

REFERENCE MAP ENTRY

```
"t": "{tinco}[triple-dot-above]"
```




The reference mark follows the explicit base in the expression and attaches to it. These two examples describe the same base-plus-mark structure.

## 2 Use the single-mode importer

**Export mode**

`one native object`

`*.tengwar-mode.json`

→

**Edit a backup; Import mode**

Validated copy gets a fresh ID

Select it and test the result

custom_modes.json is a saved collection, not a single-mode import. JSONC sources and seeded files beside the executable do not automatically replace the active built-in mode.

Keep source text, canvas adjustments and font-studio work separately. A mode definition stores writing conventions, not the current document or a font file.

VISUAL E

# Core base tokens at a glance

Use these exact native token spellings in token or baseToken. The shapes below are the same ones shown in the illustrated user guide; their linguistic values depend on the mode.

| | | | |
|---|---|---|---|
|  tinco |  parma |  calma |  quesse |
|  ando |  umbar |  anga |  ungwe |
|  thuule |  formen |  harma |  hwesta |
|  anto |  ampa |  anca |  unque |
|  nuumen |  malta |  noldo |  nwalme |
|  oore |  vala |  anna |  vilya |
|  roomen |  arda |  lambe |  alda |
|  silme |  silme-nuquerna |  esse |  esse-nuquerna |
|  hyarmen |  hwesta-sindarinwa |  yanta |  uure |

This is a visual key to 36 core bases, not the complete registry. The following catalogue retains all 121 registered base names, mark tokens and reference-expression aliases.

| Symptom | What to check |
|---|---|
| A reference group disappears | Misspelt expression token or plain text used as a map value |
| File changes beside the app do nothing | Seeded sources are not live-loaded mode definitions |

Native validation rejects duplicate association inputs, whitespace/uppercase inputs, unknown native tokens, invalid scopes, empty before/after context, invalid mark directions/offsets, and inconsistent no-output entries. Disabled entries are still structurally validated. Use proper JSON numbers and booleans; strings such as `"false"` are not converted. Unknown top-level/nested native fields such as `dy` can be silently dropped, so do not rely on them.

For contributors, run the existing targeted tests from the Flutter project root:

```
flutter test test/custom_mode_test.dart test/mode_templates_test.dart test/silent_association_test.dart
test/mode_storage_test.dart test/arabizi_release_test.dart
```

The command requires an installed Flutter SDK and the project's dependencies. Add regression cases for the new behaviour rather than relying solely on a JSON parse. Useful assertions compare base tokens, mark tokens, anchors, offsets and JSON round trips; test numeral escapes through `EditorController`.

# 12 Glyph token catalogue

Tokens are case-sensitive and refer to the bundled Alcarin outlines. Private-use font addresses are internal implementation details, not standard Unicode Tengwar assignments. Several token names can share an outline. A mode chooses the linguistic value; changing a token does not load a new font.

## Base tokens

121 registered base names. All are accepted wherever a native base token is required.

| Token | Token | Token | Token |
|---|---|---|---|
| ayin | tinco | parma | calma |
| quesse | ando | umbar | anga |
| ungwe | thuule | formen | harma |
| hwesta | anto | ampa | anca |
| unque | nuumen | malta | noldo |
| nwalme | oore | vala | anna |
| vilya | extended-tinco | extended-parma | extended-calma |
| extended-quesse | extended-ando | extended-umbar | extended-anga |
| extended-ungwe | roomen | arda | lambe |
| alda | silme | silme-nuquerna | esse |
| esse-nuquerna | hyarmen | hwesta-sindarinwa | yanta |
| uure | aara | halla | telco |
| osse | comma | stop | doubleStop |
| tripleStop | exclamation | question | semicolon |
| the-shorthand | of-shorthand | of-the-shorthand | and-shorthand |

| Token | Token | Token | Token |
|---|---|---|---|
| malta-with-curl | at-shorthand | one | reversed-osse |
| bombadil-w | ligating-short-carrier | open-anna | christopher-qu |
| bombadil-hw | mh | beleriandic-mh | lowdham-hw |
| vaiya | pusta | double-pusta | triple-pusta |
| quadruple-pusta | quintuple-pusta | parenthesis | section-mark |
| double-section-mark | left-quote | right-quote | thorin-exclamation |
| thorin-exclamation-down | thorin-exclamation-left | thorin-exclamation-right | zero |
| two | three | four | five |
| six | seven | eight | nine |
| ten | eleven | twelve | uure-with-slash |
| uure-with-slash-alt | long-carrier-alt | osse-with-tick | fronrian-yanta |
| small-lambe | thorin-equal | thorin-therefore | thorin-then |
| thorin-next | thorin-colon | thorin-semicolon | rumilian-zero |
| rumilian-one | rumilian-two | rumilian-three | rumilian-four |
| rumilian-five | rumilian-six | rumilian-seven | rumilian-eight |
| rumilian-nine | | | |

## Mark tokens

The anchor column shows the glyph library default used by reference expressions. Native JSON still needs an explicit `direction` for any default other than 8.

| Native token | Reference default anchor |
|---|---|
| a | 8 |
| threeDotsBelow | 2 |
| twoDotsAbove | 8 |
| twoDotsBelow | 2 |
| i | 8 |
| dotBelow | 2 |
| e | 8 |
| acuteBelow | 2 |
| o | 8 |
| rightCurlBelow | 2 |
| u | 8 |
| leftCurlBelow | 2 |
| ring | 8 |
| wave | 8 |

| Native token | Reference default anchor |
|---|---|
| nasalBar | 8 |
| doubleBar | 2 |
| tilde | 8 |
| y | 8 |
| breve | 8 |
| breveBelow | 2 |
| yantaAbove | 8 |
| doubleE | 8 |
| doubleO | 8 |
| doubleU | 8 |
| barInside | 5 |
| grave | 8 |
| thinnas | 8 |
| finalS | 6 |
| finalSUp | 6 |
| dotInside | 5 |
| doubleAcute | 8 |
| doubleAcuteBelow | 2 |
| reversedThreeDotsAbove | 8 |
| reversedThreeDotsBelow | 2 |
| finalSCurl2 | 6 |
| finalSCurl3 | 6 |
| finalSCurl4 | 6 |
| rightCurlBelowRight | 2 |
| leastSignificantDigit | 8 |
| doubleDotInside | 5 |
| ringBelow | 2 |
| waveBelow | 2 |
| tildeBelow | 2 |
| overTwist | 8 |
| doubleEBelow | 2 |
| doubleOBelow | 2 |
| doubleUBelow | 2 |

Legacy native aliases remain accepted: `dot` → `i`, `tripleDot` → `a`, `bar` → `doubleBar`, `finalSCurl` → `finalS`, and `curl` → `o`. Prefer the current names when authoring a new file.

### Reference expression aliases

Use these inside square brackets in `referenceData` expressions. They are not alternate native JSON token names.

| Reference spelling | Native token |
|---|---|
| `[triple-dot-above]` | `a` |
| `[reversed-triple-dot-above]` | `reversedThreeDotsAbove` |
| `[acute]` | `e` |
| `[dot-above]` | `i` |
| `[right-curl]` | `o` |
| `[left-curl]` | `u` |
| `[double-dot-above]` | `twoDotsAbove` |
| `[double-dot-below]` | `twoDotsBelow` |
| `[double-dot-inside]` | `doubleDotInside` |
| `[dot-below]` | `dotBelow` |
| `[double-acute]` | `doubleAcute` |
| `[double-left-curl]` | `doubleU` |
| `[bar-above]` | `nasalBar` |
| `[bar-below]` | `doubleBar` |
| `[bar-inside]` | `barInside` |
| `[right-curl-below]` | `rightCurlBelow` |
| `[over-twist]` | `overTwist` |
| `[yanta-above]` | `yantaAbove` |
| `[tilde-above]` | `tilde` |
| `[hook]` | `finalS` |

Registered native mark spellings, such as `[a]` and `[nasalBar]`, also pass through the reference interpreter. Prefer the bundled source conventions when editing a reference map.

## 13 Source map and further reading

Paths below are relative to the Flutter project root, the parent of this `docs` directory.

| Source | Responsibility |
|---|---|
| `lib/models/custom_mode.dart` | Native serialisation, defaults, association/rule selection and scopes |
| `lib/services/custom_mode_store.dart` | Import validation, packaged defaults, storage, backups and migration |
| `lib/engine/custom_mode_engine.dart` | Engine selection and Empty-mode transcription |
| `lib/engine/silent_input.dart` | No-output preprocessing and input classification |

| Source | Responsibility |
| --- | --- |
| `lib/engine/mode_templates.dart` | Duplication, inherited entries and reference snapshots |
| `lib/engine/english_mode.dart` | English order of processing and named rule groups |
| `lib/engine/english_readings.dart` | English word/spelling exceptions |
| `lib/engine/hebrew_mode.dart` | Hebrew built-in conventions |
| `lib/engine/reference_mode.dart` | Supported reference expressions, preprocessing and exceptions |
| `lib/engine/reference_mode_data.dart` | Embedded comment-free definitions |
| `lib/glyphs/glyph_library.dart` | Accepted base/mark tokens, aliases and default mark positions |
| `lib/engine/layout_engine.dart` | Generated glyph layout |
| `lib/controllers/editor_controller.dart` | Digits, newlines, canvas edits and source ranges |
| `lib/widgets/custom_mode_editor.dart` | Graphical mode editing |
| `lib/main.dart` | Single-mode import/export and fresh imported IDs |

Reference definitions use map, preprocess and words. The supported syntax and behaviour are documented in section 9. Follow Lambe's source when maintaining or extending a mode.

Retain the applicable source/font notices when redistributing bundled material. This reference documents Lambe 0.9.6; application version, custom-mode `schemaVersion`, collection `version` and editor-document version are separate values.